Session 1: Setup Environment & Writing First Tests

1.1 What is Playwright?

Playwright is an open-source end-to-end testing framework developed by Microsoft. It supports automation of Chromium, Firefox, and WebKit browsers using a single API. Unlike Selenium, Playwright has built-in auto-waiting, meaning it automatically waits for elements to be actionable before interacting with them.

Key Advantages Over Selenium:

- **Auto-waiting:** No Thread.sleep() or explicit waits needed. Playwright waits for element to be visible, stable, and enabled automatically.
- **Speed:** Uses Chrome DevTools Protocol (CDP) directly — significantly faster than Selenium WebDriver.
- **API Testing:** Has built-in APIRequestContext — no need for Postman or RestAssured for API tests.
- **Cross-browser:** Test on Chrome, Firefox, Safari (WebKit) with the same test script.
- **Parallel Execution:** Tests run in parallel across files by default using worker processes.
- **Tracing & Debugging:** Built-in trace viewer, video recording, and screenshot capture.

1.2 Installation Steps

1. Install Node.js (v16 or above) from nodejs.org
2. Create a new project folder and initialize it:

```
mkdir finsecure-playwright
cd finsecure-playwright
npm init -y
```

3. Install Playwright with TypeScript support:

```
npm init playwright@latest
# During setup, choose:
# > TypeScript
# > tests/ (test folder name)
# > Add GitHub Actions? Yes
# > Install browsers? Yes
```

4. Verify installation and run the sample tests:

```
npx playwright test
npx playwright show-report
```

1.3 Project Folder Structure (Banking Project)

```
finsecure-playwright/
├── tests/
│   ├── login.spec.ts          # Login test cases
│   ├── fund-transfer.spec.ts  # Transfer module tests
│   ├── loan.spec.ts           # Loan module tests
│   └── api/
│       └── transfers-api.spec.ts
├── pages/                     # POM page classes
│   ├── LoginPage.ts
│   ├── DashboardPage.ts
│   └── FundTransferPage.ts
├── fixtures/
│   └── auth.fixture.ts        # Custom fixtures for auth
├── utils/
│   └── helpers.ts             # Reusable utility functions
├── test-data/
│   ├── users.json
│   └── transfers.csv
```

```
├── playwright.config.ts       # Main configuration
└── package.json
```

1.4 playwright.config.ts — Complete Explanation

```
import { defineConfig, devices } from '@playwright/test';

export default defineConfig({
  testDir: './tests',         // Where tests are located
  timeout: 30000,             // 30s max per test
  retries: 2,                 // Retry failed tests 2 times (CI)
  workers: 4,                 // 4 parallel workers
  fullyParallel: true,        // Run tests in each file in parallel
  reporter: 'html',           // HTML reporter (also: 'json', 'junit')

  use: {
    baseURL: 'https://finsecure.sit.internal',  // Base URL for all goto()
    headless: true,                              // No browser UI in CI
    screenshot: 'only-on-failure',              // Screenshot on failure
    video: 'retain-on-failure',                 // Video on failure
    trace: 'on-first-retry',                    // Trace on retry
    actionTimeout: 10000,                       // 10s for each action
    navigationTimeout: 15000,                   // 15s for page navigation
  },

  projects: [
    { name: 'chromium', use: { ...devices['Desktop Chrome'] } },
    { name: 'firefox',  use: { ...devices['Desktop Firefox'] } },
    { name: 'webkit',   use: { ...devices['Desktop Safari'] } },
    { name: 'mobile',   use: { ...devices['iPhone 13'] } },
  ],
});
```

> **Interview Tip:** Mention that baseURL allows you to switch SIT/UAT/PROD environments using environment variables — no hardcoding URLs in individual tests.

1.5 Writing Your First Test

```
// tests/login.spec.ts
import { test, expect } from '@playwright/test';

test('valid login should navigate to dashboard', async ({ page }) => {
  await page.goto('/login');                          // Uses baseURL from config
  await page.fill('#username', 'testuser01');         // Fill username
  await page.fill('#password', 'Pass@123');           // Fill password
  await page.click('button[type="submit"]');         // Click login

  // Assertions
  await expect(page).toHaveURL(/dashboard/);
  await expect(page.getByText('Welcome, Test User')).toBeVisible();
});
```

Session 2: Playwright Built-In Locators

2.1 What are Locators?

Locators in Playwright are the primary way to find and interact with elements on the page. Unlike Selenium, Playwright locators are lazy — they don't actually search the DOM when you create them. The search happens only when you perform an action (click, fill, etc.) and they automatically retry until the element is found or timeout occurs.

2.2 getByRole — Best Locator (Accessibility-Based)

getByRole finds elements by their ARIA role. This is the most recommended locator as it reflects how real users and screen readers see the page.

```
// Button by role
await page.getByRole('button', { name: 'Transfer Funds' }).click();

// Link by role
await page.getByRole('link', { name: 'Account Statement' }).click();

// Input/textbox by role
await page.getByRole('textbox', { name: 'Amount' }).fill('5000');

// Heading
await expect(page.getByRole('heading', { name: 'Fund Transfer' })).toBeVisible();

// Checkbox
await page.getByRole('checkbox', { name: 'Save Beneficiary' }).check();

// Dropdown/combobox
await page.getByRole('combobox', { name: 'Transfer Type' }).selectOption('IMPS');
```

2.3 getByText — Find by Visible Text

```
// Exact text match
await page.getByText('Transfer Successful').click();

// Partial text match (substring)
await page.getByText('Welcome').click();

// Exact match using option
await page.getByText('Transfer Successful', { exact: true }).click();

// Banking scenario: verify success message
await expect(page.getByText('Your IMPS transfer was successful')).toBeVisible();
```

2.4 getByLabel — Best for Form Fields

```
// Finds input associated with <label> text
await page.getByLabel('Account Number').fill('1234567890');
await page.getByLabel('IFSC Code').fill('SBIN0001234');
await page.getByLabel('Transfer Amount').fill('10000');

// Works even with 'for' attribute on label
// <label for='amt'>Transfer Amount</label>
// <input id='amt' />
await page.getByLabel('Transfer Amount').fill('5000');
```

2.5 getByPlaceholder — For Inputs With Placeholder Text

```
await page.getByPlaceholder('Enter your username').fill('testuser01');
await page.getByPlaceholder('Enter OTP').fill('123456');
await page.getByPlaceholder('Search transactions...').fill('NEFT');
```

2.6 getByTestId — Most Stable Locator

data-testid attributes are added by developers specifically for testing. They never change with UI redesigns, making them the most stable locator strategy.

```
// HTML: <button data-testid='transfer-submit-btn'>Submit</button>
await page.getByTestId('transfer-submit-btn').click();

// HTML: <input data-testid='beneficiary-account' />
await page.getByTestId('beneficiary-account').fill('9876543210');
```

> **Interview Tip:** Always prefer locators in this priority order: getByTestId > getByRole > getByLabel > getByPlaceholder > getByText > CSS > XPath. Role-based locators are closest to how users interact with the app.

Session 3 & 4: XPath Locators and XPath Axes

3.1 What is XPath?

XPath (XML Path Language) is a query language to navigate and select nodes in an XML/HTML document. It is useful when built-in Playwright locators are not sufficient — for example, to traverse relationships between elements (parent, sibling, ancestor, following).

3.2 Absolute vs Relative XPath

```
// Absolute XPath — fragile, breaks on any HTML change
// NEVER use in automation
/html/body/div[2]/form/div[1]/input

// Relative XPath — starts with //, recommended
//input[@id='username']
//button[@type='submit']
//h1[text()='Fund Transfer']
```

3.3 XPath Syntax Patterns

```
// By attribute
//input[@id='username']
//input[@name='amount']
//button[@class='btn-primary']

// By text
//button[text()='Transfer Funds']
//span[contains(text(), 'Welcome')]

// By multiple attributes (AND)
//input[@type='text' and @name='account']

// By index (use carefully)
(//input[@type='text'])[1]   // first text input on page
(//tr)[3]                    // third table row
```

3.4 XPath Axes — Navigate DOM Relationships

parent:: — Go to parent element

```
// Find the label's parent div
//label[text()='Account Number']/parent::div

// Banking: find the form containing the transfer button
//button[text()='Submit']/parent::form
```

ancestor:: — Go up multiple levels

```
// Find table row containing a specific cell
//td[text()='IMPS']/ancestor::tr

// Find the section containing a particular field
//input[@id='amount']/ancestor::div[@class='transfer-section']
```

following-sibling:: — Next sibling elements

```
// Get error message that appears after an invalid input field
//input[@id='account-number']/following-sibling::span[@class='error-msg']

// Get the next cell in the same table row
//td[text()='Account Holder']/following-sibling::td
```

preceding-sibling:: — Previous sibling elements

```
// Find the label before an input
//input[@id='amount']/preceding-sibling::label
```

child:: — Direct children

```
//form[@id='transfer-form']/child::input
//ul[@id='nav-menu']/child::li
```

descendant:: — All nested children

```
// All buttons inside the transfer modal
//div[@id='transfer-modal']/descendant::button
```

following:: — All elements after current node in DOM

```
// All elements after the 'Transfer Amount' field
//input[@id='amount']/following::button
```

> **Interview Tip:** XPath axes are commonly asked in interviews. Practice explaining parent, ancestor, following-sibling with a real DOM scenario. Axis = direction of traversal in DOM tree.

Session 5: CSS Locators

5.1 CSS Selector Fundamentals

CSS selectors are faster than XPath in most browsers and are easier to read. They select HTML elements based on their tag, class, ID, attributes, and position.

```
// By ID
await page.locator('#username').fill('testuser01');

// By class
await page.locator('.btn-transfer').click();

// By tag + class
await page.locator('button.btn-primary').click();

// By attribute
await page.locator('input[type="password"]').fill('Pass@123');
await page.locator('input[name="amount"]').fill('5000');

// Attribute contains value (substring)
await page.locator('div[class*="alert"]');
```

```
// Attribute starts with value
await page.locator('input[id^="transfer"]');

// Attribute ends with value
await page.locator('input[id$="-amount"]');
```

5.2 CSS Combinators

```
// Descendant (space) — any nested child
await page.locator('.transfer-form input');

// Direct child (>) — only immediate child
await page.locator('form > div > input');

// Adjacent sibling (+) — immediately next sibling
await page.locator('label + input');

// General sibling (~) — any following sibling
await page.locator('input ~ span.error');
```

5.3 CSS Pseudo-classes

```
// nth-child — select by position
await page.locator('tr:nth-child(2)');       // 2nd row
await page.locator('tr:nth-child(odd)');     // odd rows
await page.locator('tr:nth-child(even)');    // even rows

// first-child and last-child
await page.locator('li:first-child');
await page.locator('li:last-child');

// not — exclude elements
await page.locator('button:not(.disabled)');

// Banking: select all transfer rows except the header
await page.locator('table tr:not(:first-child)');
```

5.4 Playwright-Specific CSS Extensions

```
// :has() — element that contains another (Playwright supports this)
// Find table row that has a cell with 'IMPS'
await page.locator('tr:has(td:text("IMPS"))');

// Combine CSS with text filter
await page.locator('button').filter({ hasText: 'Transfer' });
```

Session 6: Playwright Actions — Handling HTML Elements

6.1 Click Actions

```
// Regular click
await page.locator('#login-btn').click();

// Double click
await page.locator('#transaction-row').dblclick();
```

```
// Right click
await page.locator('#account-card').click({ button: 'right' });

// Click at specific coordinates
await page.locator('#chart').click({ position: { x: 100, y: 50 } });

// Click with modifier (Ctrl+Click for multi-select)
await page.locator('#item').click({ modifiers: ['Control'] });

// Force click (bypasses actionability checks - use carefully)
await page.locator('#hidden-btn').click({ force: true });
```

6.2 Input Actions

```
// Fill — clears existing text and types
await page.locator('#amount').fill('50000');

// Type — types character by character (triggers keydown events)
await page.locator('#amount').pressSequentially('50000', { delay: 100 });

// Clear field
await page.locator('#amount').clear();

// Press a key
await page.locator('#search').press('Enter');
await page.locator('#form').press('Tab');

// Keyboard shortcut
await page.keyboard.press('Control+A');
await page.keyboard.press('Control+C');
```

6.3 Checkbox and Radio

```
// Check a checkbox
await page.getByLabel('Save beneficiary for future').check();

// Uncheck
await page.getByLabel('Receive SMS alerts').uncheck();

// Check only if unchecked
const cb = page.getByLabel('Accept Terms');
if (!(await cb.isChecked())) { await cb.check(); }

// Radio button
await page.getByLabel('IMPS').check();    // Select IMPS transfer type
```

6.4 Hover and Focus

```
// Hover to trigger tooltip or dropdown
await page.locator('#account-menu').hover();

// Focus on element (useful for keyboard testing)
await page.locator('#otp-input').focus();

// Tap (for mobile emulation)
await page.locator('#mobile-menu-icon').tap();
```

6.5 File Upload

```
// Upload a single file
await page.locator('#upload-statement').setInputFiles('./test-data/statement.pdf');

// Upload multiple files
await page.locator('#docs').setInputFiles(['./doc1.pdf', './doc2.pdf']);

// Clear file input
await page.locator('#upload').setInputFiles([]);
```

Session 7 & 8: Handling Dropdowns and Select Options

7.1 HTML <select> Dropdowns

```
// Select by value attribute
await page.locator('#transfer-type').selectOption('IMPS');

// Select by visible text
await page.locator('#transfer-type').selectOption({ label: 'National Electronic Fund Transfer'
});

// Select by index (0-based)
await page.locator('#transfer-type').selectOption({ index: 2 });

// Multi-select (for multi-select dropdowns)
await page.locator('#account-filter').selectOption(['savings', 'current']);

// Banking scenario: Select bank from dropdown in beneficiary form
await page.getByLabel('Select Bank').selectOption({ label: 'State Bank of India' });
```

7.2 Custom Dropdowns (Non-<select> Elements)

Many modern banking UIs use custom dropdowns built with divs, ul/li instead of native select. These need a different approach.

```
// Step 1: Click the dropdown trigger to open it
await page.locator('.transfer-type-dropdown').click();

// Step 2: Wait for options to appear
await page.locator('.dropdown-menu').waitFor({ state: 'visible' });

// Step 3: Click the desired option
await page.getByRole('option', { name: 'RTGS' }).click();

// OR — find and click the option by text
await page.locator('.dropdown-item').filter({ hasText: 'RTGS' }).click();

// Verify selection
await expect(page.locator('.transfer-type-dropdown')).toHaveText('RTGS');
```

7.3 Bootstrap / jQuery UI Dropdowns

```
// jQuery UI Select2 dropdown — common in banking admin panels
// Step 1: Click the select2 trigger
await page.locator('.select2-selection').click();

// Step 2: Type to search
```

```
await page.locator('.select2-search__field').fill('HDFC');

// Step 3: Select from results
await page.locator('.select2-results__option').filter({ hasText: 'HDFC Bank' }).click();
```

Session 9 & 10: Extract Text, Web Tables, Dynamic & Pagination Tables

9.1 Extracting Text from Elements

```
// Get text content of a single element
const balance = await page.locator('#account-balance').textContent();
console.log(balance); // '₹1,25,000.00'

// Get inner text (excludes hidden text)
const welcomeMsg = await page.locator('.welcome-banner').innerText();

// Get value of input field
const enteredAmount = await page.locator('#amount').inputValue();

// Get all texts from multiple elements
const txnTypes = await page.locator('td.txn-type').allTextContents();
console.log(txnTypes); // ['IMPS', 'NEFT', 'RTGS', 'UPI']

// Get attribute value
const href = await page.locator('a#statement-link').getAttribute('href');
const isDisabled = await page.locator('#submit-btn').getAttribute('disabled');
```

9.2 Handling Static Web Tables

```
// Banking scenario: Transaction history table
// <table id='txn-table'>
//   <tr><th>Date</th><th>Type</th><th>Amount</th><th>Status</th></tr>
//   <tr><td>01/04</td><td>IMPS</td><td>5000</td><td>Success</td></tr>
// </table>

// Count rows (excluding header)
const rowCount = await page.locator('#txn-table tbody tr').count();
console.log('Total transactions:', rowCount);

// Get text of a specific cell (row 1, column 3 = Amount)
const amount = await page.locator('#txn-table tbody tr:nth-child(1) td:nth-
child(3)').textContent();

// Iterate all rows and print each row's data
const rows = page.locator('#txn-table tbody tr');
const count = await rows.count();
for (let i = 0; i < count; i++) {
  const row = rows.nth(i);
  const cells = await row.locator('td').allTextContents();
  console.log(`Row ${i+1}:`, cells);
}
```

9.3 Finding Specific Row in Table

```
// Banking: Find row where transfer type is 'RTGS' and click View Details
const rows = page.locator('table tbody tr');
```

```
const count = await rows.count();

for (let i = 0; i < count; i++) {
  const typeCell = rows.nth(i).locator('td:nth-child(2)');
  const typeText = await typeCell.textContent();
  if (typeText?.trim() === 'RTGS') {
    await rows.nth(i).locator('button.view-details').click();
    break;
  }
}
```

9.4 Dynamic Tables with Pagination

```
// Banking: Extract all transactions across multiple pages
const allTxns: string[][] = [];

while (true) {
  // Get all rows on current page
  const rows = page.locator('table tbody tr');
  const count = await rows.count();

  for (let i = 0; i < count; i++) {
    const cells = await rows.nth(i).locator('td').allTextContents();
    allTxns.push(cells);
  }

  // Check if Next button is enabled
  const nextBtn = page.locator('button#next-page');
  const isDisabled = await nextBtn.getAttribute('disabled');
  if (isDisabled !== null) break; // No more pages

  await nextBtn.click();
  await page.waitForLoadState('networkidle');
}

console.log('Total transactions across all pages:', allTxns.length);
```

Session 11: Handle jQuery & Bootstrap Date Pickers

11.1 Native Date Input

```
// HTML5 date input: <input type='date' id='txn-date'>
await page.locator('#txn-date').fill('2025-04-01'); // Format: YYYY-MM-DD

// Verify the date was set
await expect(page.locator('#txn-date')).toHaveValue('2025-04-01');
```

11.2 jQuery UI Datepicker

```
// Banking: Select transaction date range
// Step 1: Click the date input to open calendar
await page.locator('#from-date').click();

// Step 2: Navigate to correct month
// Click Next arrow to go to next month
await page.locator('.ui-datepicker-next').click();

// Step 3: Select the day
```

```
await page.locator('.ui-datepicker-calendar td a').filter({ hasText: '15' }).click();

// Practical approach: directly set value via JS (most reliable)
await page.evaluate(() => {
  const input = document.querySelector('#from-date') as HTMLInputElement;
  input.value = '01/04/2025';
  // Trigger change event so jQuery picks up the change
  input.dispatchEvent(new Event('change', { bubbles: true }));
});
```

11.3 Bootstrap Datepicker

```
// Step 1: Click input to open
await page.locator('#loan-start-date').click();

// Step 2: Click month/year header to go to month view
await page.locator('.datepicker-switch').click();

// Step 3: Click year header to go to year view
await page.locator('.datepicker-switch').click();

// Step 4: Select year
await page.locator('span.year').filter({ hasText: '2025' }).click();

// Step 5: Select month
await page.locator('span.month').filter({ hasText: 'Apr' }).click();

// Step 6: Select day
await page.locator('td.day').filter({ hasText: '15' }).first().click();
```

Session 12: Dialogs (Alert, Confirm, Prompt), Frames & Inner Frames

12.1 Handling JavaScript Alerts

```
// Alert — has only OK button
// Must set up handler BEFORE the action that triggers alert
page.on('dialog', async dialog => {
  console.log('Alert message:', dialog.message());
  await dialog.accept(); // Click OK
});
await page.locator('#delete-account-btn').click(); // This triggers alert
```

12.2 Confirm Dialog

```
// Confirm — has OK and Cancel
page.on('dialog', async dialog => {
  const msg = dialog.message();
  console.log('Confirm:', msg); // 'Are you sure you want to transfer ₹50,000?'
  await dialog.accept();   // OK — confirm transfer
  // await dialog.dismiss(); // Cancel — reject transfer
});
await page.locator('#confirm-transfer-btn').click();
```

12.3 Prompt Dialog

```
// Prompt — has text input, OK, and Cancel
```

```
page.on('dialog', async dialog => {
  await dialog.accept('testuser01'); // Type value and click OK
});
await page.locator('#enter-username-btn').click();
```

12.4 Frames (iframes)

iframes are embedded HTML documents within the main page. In banking portals, payment gateways (like CCAvenue, Razorpay) often render inside iframes. You must switch into the frame to interact with its content.

```
// Find frame by name attribute
const frame = page.frame({ name: 'payment-gateway-frame' });

// Find frame by URL pattern
const frame2 = page.frameLocator('iframe[src*="razorpay"]');

// Interact with elements inside the frame
await frame2.locator('#card-number').fill('4111 1111 1111 1111');
await frame2.locator('#expiry').fill('12/26');
await frame2.locator('#cvv').fill('123');
await frame2.locator('button#pay-now').click();
```

12.5 Nested Frames (Frame within Frame)

```
// Outer frame -> inner frame -> element
const outerFrame = page.frameLocator('#payment-wrapper');
const innerFrame = outerFrame.frameLocator('#secure-input-frame');
await innerFrame.locator('#card-cvv').fill('123');
```

Session 13: Browser Context, Tabs & Pages / Popups

13.1 Browser Context

A BrowserContext is an isolated browser session — like an incognito window. It has its own cookies, localStorage, and sessions. Multiple contexts can run simultaneously without sharing state.

```
// Create a new browser context
const context = await browser.newContext();
const page = await context.newPage();
await page.goto('/login');

// Save auth state to file (for reuse across tests)
await context.storageState({ path: './auth-state.json' });

// Load saved auth state in another test
const authContext = await browser.newContext({
  storageState: './auth-state.json'  // Already logged in!
});
```

> **Interview Tip:** In banking automation, save the logged-in session state and reuse it across tests to avoid logging in before every single test — saves significant execution time.

13.2 Handling Multiple Tabs

```
// Scenario: Clicking 'Open Statement' opens a new tab in FinSecure

// Wait for the new page (tab) to open
```

```
const [newPage] = await Promise.all([
  context.waitForEvent('page'),          // Wait for new tab
  page.locator('#open-statement-btn').click() // Trigger the new tab
]);

// Wait for new tab to load
await newPage.waitForLoadState();

// Interact with the new tab
await expect(newPage).toHaveURL(/statement/);
const statementTitle = await newPage.locator('h1').textContent();
console.log('Statement page title:', statementTitle);

// Close the new tab and return to original
await newPage.close();
```

13.3 Handling Popup Windows

```
// Popup (new window) — same as new tab handling
const [popup] = await Promise.all([
  page.waitForEvent('popup'),
  page.locator('#terms-link').click()
]);

await popup.waitForLoadState();
const termsText = await popup.locator('h1').textContent();
await popup.close();
```

13.4 Simulating Two Different Users (Multi-Context)

```
// Banking scenario: Test transfer between two accounts
const senderContext = await browser.newContext({ storageState: './sender-auth.json' });
const receiverContext = await browser.newContext({ storageState: './receiver-auth.json' });

const senderPage = await senderContext.newPage();
const receiverPage = await receiverContext.newPage();

// Sender initiates transfer
await senderPage.goto('/transfer');
// ... perform transfer ...

// Receiver checks balance (parallel)
await receiverPage.goto('/account-summary');
// ... verify credit ...
```

Session 14: Auto-Waiting, Timeouts, Assertions & Codegen

14.1 How Auto-Waiting Works

Every Playwright action (click, fill, check, etc.) automatically waits for the element to be: (1) Attached to DOM, (2) Visible, (3) Stable (not animating), (4) Enabled (not disabled), (5) Editable (for fill). This eliminates most flaky test issues caused by timing.

```
// Playwright auto-waits — no need for explicit waits in most cases
await page.locator('#submit-btn').click(); // Waits until button is clickable
await page.locator('#amount').fill('5000'); // Waits until input is editable

// waitFor — explicit wait for a specific state
```

```
await page.locator('#success-message').waitFor({ state: 'visible' });
await page.locator('#loading-spinner').waitFor({ state: 'hidden' });
await page.locator('#transaction-table').waitFor({ state: 'attached' });
```

14.2 Types of Waits

```
// 1. waitForURL — wait until URL matches pattern
await page.waitForURL('**/dashboard');
await page.waitForURL(/login/);

// 2. waitForLoadState
await page.waitForLoadState('load');            // HTML loaded
await page.waitForLoadState('domcontentloaded'); // DOM ready
await page.waitForLoadState('networkidle');     // No network requests for 500ms

// 3. waitForResponse — wait for specific API response
const [response] = await Promise.all([
  page.waitForResponse(res => res.url().includes('/api/transfer')),
  page.locator('#submit-btn').click()
]);
const body = await response.json();
console.log('Transfer API response:', body);
```

14.3 Assertions (expect)

```
// Page-level assertions
await expect(page).toHaveURL(/dashboard/);
await expect(page).toHaveTitle('FinSecure — Dashboard');

// Element visibility
await expect(page.locator('#success-alert')).toBeVisible();
await expect(page.locator('#error-msg')).toBeHidden();

// Text content
await expect(page.locator('#balance')).toHaveText('₹1,25,000.00');
await expect(page.locator('#status')).toContainText('Success');

// Input value
await expect(page.locator('#amount')).toHaveValue('5000');

// Attribute
await expect(page.locator('#submit-btn')).toBeEnabled();
await expect(page.locator('#old-feature')).toBeDisabled();
await expect(page.locator('#checkbox')).toBeChecked();

// Count
await expect(page.locator('table tbody tr')).toHaveCount(10);

// Soft assertions — don't stop test on failure
await expect.soft(page.locator('#name')).toHaveText('John');
await expect.soft(page.locator('#email')).toHaveText('john@test.com');
// Test continues even if above assertions fail
```

14.4 Codegen — Auto-Generate Tests

```
// Record your actions and generate test code automatically
npx playwright codegen https://finsecure.sit.internal

// Record with auth state
```

```
npx playwright codegen --save-storage=auth.json https://finsecure.sit.internal

// Record with specific device
npx playwright codegen --device='iPhone 13' https://finsecure.sit.internal
```

Note: Codegen is great for quickly generating locators. But always review and refactor the generated code — raw codegen output uses CSS selectors and doesn't follow POM pattern.

Session 15: Trace Viewer, Screenshots, Videos & Flaky Tests

15.1 Taking Screenshots

```
// Full page screenshot
await page.screenshot({ path: './screenshots/dashboard.png', fullPage: true });

// Screenshot of a specific element only
await page.locator('#transaction-table').screenshot({ path: './screenshots/table.png' });

// Screenshot on test failure (automatic via config)
// playwright.config.ts: screenshot: 'only-on-failure'

// Embed screenshot in test report
const buffer = await page.screenshot();
await testInfo.attach('Dashboard Screenshot', {
  body: buffer,
  contentType: 'image/png'
});
```

15.2 Video Recording

```
// playwright.config.ts
use: {
  video: 'on',                 // Always record
  video: 'off',                // Never record
  video: 'retain-on-failure',  // Only keep video if test fails
  video: 'on-first-retry',     // Record during first retry
}
```

15.3 Trace Viewer

The Trace Viewer is Playwright's most powerful debugging tool. It records a full snapshot of the test — every action, screenshot, network request/response, console logs, and DOM state at each step.

```
// playwright.config.ts
use: { trace: 'on-first-retry' }  // Capture trace on first retry only

// Run tests and view trace
npx playwright test --trace on
npx playwright show-trace trace.zip

// Attach trace in test
await testInfo.attach('trace', {
  path: testInfo.outputPath('trace.zip'),
  contentType: 'application/zip'
});
```

Interview Tip: When debugging a failing test, open the trace viewer — it shows exactly what the page looked like at every step, which network calls were made, and what the DOM contained. This is how you explain root cause analysis in interviews.

15.4 Handling Flaky Tests

A flaky test is one that sometimes passes and sometimes fails without any code change — usually caused by timing, dynamic content, or network latency.

- **Cause 1 — Race conditions:** Element appears briefly and disappears. Fix: use waitFor({ state: 'visible' }) before asserting.
- **Cause 2 — Hardcoded waits:** page.waitForTimeout(2000) is fragile. Use event-based waits instead.
- **Cause 3 — Order dependency:** Tests sharing state. Fix: use fixtures and isolated contexts.
- **Fix — retries:** Set retries: 2 in config. Use test.retries(2) for specific test.

```
// Mark test as flaky — will report separately
test('flaky login test', async ({ page }) => {
  test.fixme(); // Skip until fixed
});

// Set retries for a specific test
test('payment gateway test', async ({ page }) => {
  test.slow(); // Triples the timeout for this test
});
```

Session 16: Grouping Tests, Hooks, Annotations & Tags

16.1 test.describe — Grouping Tests

```
import { test, expect } from '@playwright/test';

test.describe('Fund Transfer Module', () => {

  test.describe('IMPS Transfers', () => {
    test('successful IMPS transfer', async ({ page }) => { /* ... */ });
    test('IMPS with insufficient funds', async ({ page }) => { /* ... */ });
  });

  test.describe('NEFT Transfers', () => {
    test('NEFT transfer queued for next batch', async ({ page }) => { /* ... */ });
  });

});
```

16.2 Hooks — beforeAll, afterAll, beforeEach, afterEach

```
test.describe('Login Tests', () => {

  // Runs ONCE before all tests in this describe block
  test.beforeAll(async ({ browser }) => {
    // Setup: create test accounts in database
    console.log('Setting up test data...');
  });

  // Runs before EACH test
  test.beforeEach(async ({ page }) => {
    await page.goto('/login');
    await page.locator('#username').fill('testuser01');
    await page.locator('#password').fill('Pass@123');
    await page.locator('#login-btn').click();
  });

  // Runs after EACH test — cleanup
```

```
  test.afterEach(async ({ page }) => {
    await page.goto('/logout');
  });

  // Runs ONCE after all tests
  test.afterAll(async () => {
    // Cleanup: delete test data
    console.log('Teardown complete');
  });

  test('valid login', async ({ page }) => { /* ... */ });
  test('session timeout', async ({ page }) => { /* ... */ });

});
```

16.3 Annotations

```
// Skip a test
test.skip('skipping until bug BNK-1042 is fixed', async ({ page }) => { });

// Skip conditionally
test('mobile only feature', async ({ page, isMobile }) => {
  test.skip(!isMobile, 'This test is for mobile only');
  // ...test code...
});

// Mark as expected to fail (known bug)
test.fail('EMI calculator bug BNK-1078', async ({ page }) => {
  // Test documents a known failure
});

// Slow test — triples timeout
test('full loan application flow', async ({ page }) => {
  test.slow();
  // ... long test ...
});
```

16.4 Tags — Filter Tests by Tag

```
// Add tags to tests
test('login — smoke test @smoke', async ({ page }) => { /* ... */ });
test('fund transfer @regression @payment', async ({ page }) => { /* ... */ });
test('loan apply @regression @loan', async ({ page }) => { /* ... */ });

// Run only smoke tests
npx playwright test --grep @smoke

// Run regression tests excluding loan module
npx playwright test --grep @regression --grep-invert @loan

// Run multiple tags
npx playwright test --grep '@smoke|@critical'
```

> **Interview Tip:** Tags are very practical in banking projects — you can run only @smoke before a release sanity check, and full @regression after each sprint deployment.

Session 17: Parallelism & Parallel Testing

17.1 How Playwright Runs Tests in Parallel

By default, Playwright runs test files in parallel (each file in its own worker process). Tests within a single file run sequentially. This is safe because each worker gets its own browser context — no shared state between parallel tests.

```
// playwright.config.ts
export default defineConfig({
  workers: 4,              // 4 parallel workers
  fullyParallel: true,     // Also parallelize tests within a file
  retries: 2,
});
```

17.2 Controlling Parallelism

```
// Run tests serially within this file (sequential order)
test.describe.serial('Sequential payment flow', () => {
  test('step 1: add beneficiary', async ({ page }) => { });
  test('step 2: initiate transfer', async ({ page }) => { });
  test('step 3: verify receipt', async ({ page }) => { });
});

// Run tests from command line with specific worker count
npx playwright test --workers=2
npx playwright test --workers=1  // Fully sequential (debug mode)
```

17.3 Sharding — Distribute Tests Across CI Machines

```
// Split tests across 3 CI machines
// Machine 1:
npx playwright test --shard=1/3
// Machine 2:
npx playwright test --shard=2/3
// Machine 3:
npx playwright test --shard=3/3

// Merge reports from all shards
npx playwright merge-reports ./all-blob-reports --reporter html
```

> **Note:** Sharding is useful in large banking test suites with 500+ tests. Instead of running all 500 sequentially, you split across multiple machines and merge results.

Session 18: Parameterization & Data-Driven Testing

18.1 Inline Data-Driven with Arrays

```
// Banking: Test multiple transfer amounts with different expected outcomes
const transferTestData = [
  { amount: '1000',    type: 'IMPS', expected: 'Success' },
  { amount: '500000',  type: 'RTGS', expected: 'Success' },
  { amount: '0',       type: 'NEFT', expected: 'Error: Amount must be greater than 0' },
  { amount: '9999999', type: 'IMPS', expected: 'Error: Exceeds daily limit' },
];

for (const data of transferTestData) {
  test(`Transfer ${data.amount} via ${data.type} — expects ${data.expected}`, async ({ page }) =>
{
    await page.goto('/transfer');
    await page.getByLabel('Amount').fill(data.amount);
    await page.getByLabel('Transfer Type').selectOption(data.type);
```

```
    await page.getByRole('button', { name: 'Submit' }).click();
    await expect(page.locator('.status-message')).toContainText(data.expected);
  });
}
```

18.2 Reading Test Data from JSON

```
// test-data/users.json
{
  "users": [
    { "username": "testuser01", "password": "Pass@123", "role": "retail" },
    { "username": "corpuser01", "password": "Corp@456", "role": "corporate" },
    { "username": "adminuser",  "password": "Admin@789", "role": "admin" }
  ]
}

// In test file:
import * as testData from '../test-data/users.json';

for (const user of testData.users) {
  test(`Login as ${user.role} user: ${user.username}`, async ({ page }) => {
    await page.goto('/login');
    await page.getByLabel('Username').fill(user.username);
    await page.getByLabel('Password').fill(user.password);
    await page.getByRole('button', { name: 'Login' }).click();
    await expect(page).toHaveURL(/dashboard/);
  });
}
```

18.3 Reading from CSV

```
// utils/csvReader.ts
import * as fs from 'fs';

export function readCSV(filePath: string): Record<string, string>[] {
  const content = fs.readFileSync(filePath, 'utf-8');
  const lines = content.trim().split('\n');
  const headers = lines[0].split(',');
  return lines.slice(1).map(line => {
    const values = line.split(',');
    return headers.reduce((obj, h, i) => ({ ...obj, [h.trim()]: values[i].trim() }), {});
  });
}

// transfers.csv
// accountNo,ifsc,amount,type
// 1234567890,SBIN0001234,5000,IMPS
// 9876543210,HDFC0001234,25000,NEFT

// In test:
const transfers = readCSV('./test-data/transfers.csv');
for (const t of transfers) {
  test(`Transfer to ${t.accountNo} via ${t.type}`, async ({ page }) => {
    // ... test using t.accountNo, t.ifsc, t.amount, t.type
  });
}
```

18.4 Reading from Excel (.xlsx)

```
// Install: npm install xlsx
import * as XLSX from 'xlsx';

function readExcel(filePath: string, sheetName: string) {
  const workbook = XLSX.readFile(filePath);
  const sheet = workbook.Sheets[sheetName];
  return XLSX.utils.sheet_to_json(sheet);
}

const loanData = readExcel('./test-data/loan-test-data.xlsx', 'LoanCases');

for (const loan of loanData as any[]) {
  test(`Loan application: ${loan.LoanType} — ${loan.Amount}`, async ({ page }) => {
    await page.goto('/loans/apply');
    await page.getByLabel('Loan Type').selectOption(loan.LoanType);
    await page.getByLabel('Amount').fill(String(loan.Amount));
    // ...
  });
}
```

Session 19: Reporters — HTML, JSON, JUnit & Allure

19.1 Built-In Reporters

```
// playwright.config.ts — configure multiple reporters
reporter: [
  ['html', { outputFolder: 'playwright-report', open: 'never' }],
  ['json', { outputFile: 'test-results/results.json' }],
  ['junit', { outputFile: 'test-results/junit.xml' }],
  ['list'],  // Console output during run
],
```

19.2 HTML Reporter

```
// Generate and open HTML report
npx playwright test
npx playwright show-report

// The HTML report includes:
// - Pass/fail status for every test
// - Screenshots on failure
// - Videos on failure
// - Trace viewer link on retry
// - Filter by status, tag, project (browser)
```

19.3 Allure Reporter — Most Detailed Report

```
// Install Allure Playwright
npm install --save-dev allure-playwright

// playwright.config.ts
reporter: [['allure-playwright']],

// Run and generate report
npx playwright test
npx allure generate allure-results --clean -o allure-report
npx allure open allure-report
```

```
// Add allure labels in tests
import { allure } from 'allure-playwright';

test('IMPS transfer success', async ({ page }) => {
  allure.label('feature', 'Fund Transfer');
  allure.label('severity', 'critical');
  allure.description('Verify successful IMPS transfer between accounts');
  // ... test steps ...
});
```

19.4 JUnit Reporter — For CI/CD Integration

JUnit XML format is universally supported by CI tools — Jenkins, GitHub Actions, GitLab CI. It shows test results directly in the pipeline dashboard without opening an HTML file.

```
// playwright.config.ts
reporter: [['junit', { outputFile: 'results/junit.xml' }]],

// GitHub Actions — display results in pipeline
# .github/workflows/playwright.yml
- name: Run Playwright tests
  run: npx playwright test

- name: Upload test results
  uses: actions/upload-artifact@v3
  with:
    name: playwright-report
    path: playwright-report/
```

Bonus: Page Object Model (POM) with TypeScript

POM — LoginPage.ts

```
// pages/LoginPage.ts
import { Page, Locator, expect } from '@playwright/test';

export class LoginPage {
  private readonly page: Page;
  private readonly usernameInput: Locator;
  private readonly passwordInput: Locator;
  private readonly loginButton: Locator;
  private readonly errorMessage: Locator;

  constructor(page: Page) {
    this.page = page;
    this.usernameInput = page.getByLabel('Username');
    this.passwordInput = page.getByLabel('Password');
    this.loginButton = page.getByRole('button', { name: 'Login' });
    this.errorMessage = page.locator('.error-message');
  }

  async goto() {
    await this.page.goto('/login');
  }

  async login(username: string, password: string) {
    await this.usernameInput.fill(username);
    await this.passwordInput.fill(password);
    await this.loginButton.click();
  }
```

```
  async getErrorMessage(): Promise<string> {
    return await this.errorMessage.textContent() ?? '';
  }

  async isOnDashboard(): Promise<boolean> {
    return this.page.url().includes('/dashboard');
  }
}
```

POM — Usage in Test File

```
// tests/login.spec.ts
import { test, expect } from '@playwright/test';
import { LoginPage } from '../pages/LoginPage';

test.describe('Login Module — FinSecure Banking', () => {

  test('valid credentials navigate to dashboard', async ({ page }) => {
    const loginPage = new LoginPage(page);
    await loginPage.goto();
    await loginPage.login('testuser01', 'Pass@123');
    await expect(page).toHaveURL(/dashboard/);
  });

  test('invalid password shows error message', async ({ page }) => {
    const loginPage = new LoginPage(page);
    await loginPage.goto();
    await loginPage.login('testuser01', 'wrongpass');
    const error = await loginPage.getErrorMessage();
    expect(error).toContain('Invalid credentials');
  });

});
```

Bonus: API Testing with Playwright

API Testing — Basic Request

```
// tests/api/transfers-api.spec.ts
import { test, expect } from '@playwright/test';

test('POST /api/transfer — successful IMPS transfer', async ({ request }) => {
  const response = await request.post('/api/v2/transfers/initiate', {
    headers: {
      'Authorization': 'Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...',
      'Content-Type': 'application/json'
    },
    data: {
      fromAccount: '1234567890',
      toAccount: '9876543210',
      amount: 5000,
      transferType: 'IMPS',
      ifsc: 'HDFC0001234',
      remarks: 'Test transfer'
    }
  });

  expect(response.status()).toBe(201);
  const body = await response.json();
```

```
  expect(body).toHaveProperty('txn_id');
  expect(body.status).toBe('SUCCESS');
  expect(response.headers()['content-type']).toContain('application/json');
});
```

API + UI Combined Test

```
// Create test data via API, then verify in UI
test('transfer created via API appears in UI history', async ({ page, request }) => {

  // Step 1: Create transfer via API
  const apiResponse = await request.post('/api/v2/transfers/initiate', {
    data: { fromAccount: '1234567890', toAccount: '9876543210', amount: 1500, type: 'NEFT' }
  });
  const { txn_id } = await apiResponse.json();

  // Step 2: Login to UI
  await page.goto('/login');
  await page.getByLabel('Username').fill('testuser01');
  await page.getByLabel('Password').fill('Pass@123');
  await page.getByRole('button', { name: 'Login' }).click();

  // Step 3: Verify transaction appears in history
  await page.goto('/transactions');
  await expect(page.getByText(txn_id)).toBeVisible();
});
```

Bonus: Fixtures — Reusable Test Setup

```
// fixtures/auth.fixture.ts
import { test as base, Page } from '@playwright/test';
import { LoginPage } from '../pages/LoginPage';
import { DashboardPage } from '../pages/DashboardPage';

type AuthFixtures = {
  authenticatedPage: Page;
};

export const test = base.extend<AuthFixtures>({
  authenticatedPage: async ({ page }, use) => {
    // Login before test
    const loginPage = new LoginPage(page);
    await loginPage.goto();
    await loginPage.login('testuser01', 'Pass@123');
    await page.waitForURL(/dashboard/);

    // Give the logged-in page to the test
    await use(page);

    // Logout after test
    await page.goto('/logout');
  }
});

// Usage in test:
import { test } from '../fixtures/auth.fixture';

test('view account balance — already logged in', async ({ authenticatedPage }) => {
  await authenticatedPage.goto('/accounts');
  await expect(authenticatedPage.locator('#balance')).toBeVisible();
```

```
});
```

Interview Questions & Answers

Common Playwright + TypeScript Interview Questions

Q1: What is the difference between page.click() and locator.click()?

> **Note:** page.click('#btn') is the old API — it takes a selector string and finds the element each time. locator('#btn').click() returns a Locator object that retries automatically on failure. Always prefer the Locator API as it is more reliable and auto-waits properly.

Q2: How does Playwright auto-wait work?

When you call any action method (click, fill, hover etc.) on a locator, Playwright automatically checks and waits for the element to be: attached to DOM, visible, stable (not animating), enabled, and editable (for fill). This retry loop runs until the action succeeds or the actionTimeout (default 30s) is reached.

Q3: How do you handle authentication in Playwright to avoid logging in before every test?

```
// 1. Run a global setup to login once and save session state
// global-setup.ts
import { chromium } from '@playwright/test';
async function globalSetup() {
  const browser = await chromium.launch();
  const page = await browser.newPage();
  await page.goto('/login');
  await page.fill('#username', 'testuser01');
  await page.fill('#password', 'Pass@123');
  await page.click('#login-btn');
  await page.context().storageState({ path: './auth-state.json' });
  await browser.close();
}
export default globalSetup;

// 2. Use saved state in all tests via config
// playwright.config.ts
globalSetup: './global-setup.ts',
use: { storageState: './auth-state.json' }
```

Q4: What is a BrowserContext and why is it important?

A BrowserContext is an isolated browser session — like an incognito window with its own cookies, localStorage, sessions, and permissions. It is important for: (1) running parallel tests safely without shared state, (2) testing multiple users simultaneously, (3) testing different roles (admin vs customer) in the same test run.

Q5: How do you capture and assert API responses in Playwright?

```
// Wait for API call triggered by a UI action and capture the response
const [response] = await Promise.all([
  page.waitForResponse(resp => resp.url().includes('/api/transfer') && resp.status() === 201),
  page.locator('#submit-btn').click()
]);
const responseBody = await response.json();
expect(responseBody.status).toBe('SUCCESS');
```

Q6: What is the difference between soft assertions and regular assertions?

Regular assertions (expect) stop the test immediately on failure. Soft assertions (expect.soft) log the failure but let the test continue running. Soft assertions are useful when you want to validate multiple fields on the same page in one test run — e.g., checking all fields on a transfer confirmation screen — so you get all failures at once instead of one at a time.

Q7: How do you run only specific tests or test suites?

```
// Run specific file
npx playwright test login.spec.ts

// Run tests with specific name
npx playwright test --grep 'IMPS transfer'

// Run tagged tests
npx playwright test --grep @smoke

// Run on specific browser only
npx playwright test --project=chromium

// Run in headed mode (see browser)
npx playwright test --headed
```

Q8: How do you implement POM in Playwright with TypeScript?

In POM, each application page has a corresponding TypeScript class. The class constructor receives the Playwright Page object and initializes all Locators as class properties using page.getByRole(), page.getByLabel() etc. Action methods like login(), submitTransfer() are defined in the class. Test files import these page classes and call action methods — no locators exist in the test file itself. This means if a UI element changes, you update only the page class, not every test.